Journal of Machine Learning Research 1 (2026) 1-29 Submitted 4/26; Published –

Sparse-Graph Representations for Scalable Grouped Fused Lasso Regression

Daniel Agyapong DA2343@NAU.EDU
School of Informatics, Computing, and Cyber Systems
Northern Arizona University
Flagstaff, AZ, USA

Michael Gowanlock
School of Informatics, Computing, and Cyber Systems
Northern Arizona University
Flagstaff, AZ, USA

Jane Marks
Department of Biological Sciences
Northern Arizona University
Flagstaff, AZ, USA

Toby Dylan Hocking
Département d'informatique
Université de Sherbrooke
Sherbrooke, QC, Canada

Editor:

Abstract

Although the statistical graph in grouped fused regression is often sparse, standard implementations still materialize full pairwise fusion operators or augmented designs, making the computational representation much denser than the model. We propose sparse-graph representations for grouped fused regression that preserve the graph edge set directly in the optimizer. For L1 fusion, an active-edge operator evaluates the same smoothed proximal-gradient fusion term using only nonzero graph edges. For L2 fusion, an active-edge augmented-design construction emits sparse triplets only for the modeled edges before solving the regularized path. These constructions preserve the main objective in the exact L1 and L2 cases while reducing the dominant fusion representation from full-pairwise graph construction to edge-linear construction. Our empirical comparisons hold the optimizer family fixed relative to reference grouped fused-regression solvers, so that the measured differences isolate graph representation rather than changes in optimization architecture. We also analyze a chain approximation L1 solver which trades graph fidelity for additional efficiency thereby slightly reducing accuracy. For the exact L1 and L2 solvers, the active-edge representation changes the dominant fusion-side construction from all-pairs dependence on $\binom{k}{2}$ candidate group pairs to edge-linear dependence on the m modeled graph edges. The chain approximation L1 solver further reduces the represented fusion graph to $k-1$ edges, so as to solve an approximate chain-fusion problem. On public grouped-regression datasets, we illustrate that the exact active-edge methods preserve predictive accuracy while reducing the dominant fusion-side representation cost from dependence on all $\binom{k}{2}$ group pairs to dependence on the m active graph edges. The largest practical efficiency gains occur for sparse graphs with many groups (above 100).

Keywords: fused lasso, grouped regression, sparse graphs, structured sparsity, first-order optimization

1 Introduction

Many regression problems contain a natural grouping variable such as patients may belong to disease subtypes, schools to districts or countries to continents (Gelman and Hill, 2007). Fitting a separate model in each group ignores shared signal, whereas fitting one global model forces all groups to share the same coefficients. Grouped fused regression addresses this middle ground by estimating one coefficient vector per group and penalizing differences between coefficient vectors along a graph of group relationships.

Let $X_g \in \mathbb{R}^{n_g \times p}$ and $y_g \in \mathbb{R}^{n_g}$ denote the data for group g, and let $\beta_g \in \mathbb{R}^p$ be that group's linear coefficient vector. For a graph $G = (V, E)$ over k groups with nonnegative edge weights w_{gh}, the estimators considered in this paper have the form

$$\min_{\beta_1,\ldots,\beta_k} \sum_{g=1}^{k} \frac{1}{2n_g} \|y_g - X_g\beta_g\|_2^2 + \lambda \sum_{g=1}^{k} \|\beta_g\|_1 + \gamma \sum_{(g,h)\in E} w_{gh}\Omega(\beta_g - \beta_h). \tag{1}$$

For the L1 fused model, $\Omega(u) = \|u\|_1$; for the L2 fused model, $\Omega(u) = \frac{1}{2}\|u\|_2^2$ after the standard augmented-design reformulation. This formulation is a direct descendant of the lasso, fused lasso, grouped and elastic-net regularization, graph-structured fused optimization, and subgroup regression models. The L1 solver uses the standard Nesterov smoothing strategy for nonsmooth penalties (Nesterov, 2005). The central computational issue is that the model may use only a sparse set of graph edges, while the solver may still construct fusion terms for all possible pairs of groups. A complete graph has $k(k-1)/2$ edges, but practical graphs are frequently chains (Hoefling, 2010), trees (Madrid Padilla et al., 2018), nearest-neighbor graphs (Madrid Padilla et al., 2020), or sparse networks informed by prior knowledge (Hallac et al., 2015). If the objective is defined on only m active edges, a solver that builds all pairwise fusion constraints and then uses zero weights for inactive edges changes the computational scaling of the problem without changing its statistical content. This matters especially as k grows. The modeling graph may be sparse, but the reference representation still scales with all $\binom{k}{2}$ group pairs.

The paper therefore evaluates graph representation as the primary algorithmic variable, while keeping the underlying optimizer family fixed. For L1 fusion, both the reference solver and the exact active-edge solver use the same smoothed proximal-gradient framework. For L2 fusion, both the reference solver and the active-edge solver use the same augmented-design reduction before fitting the regularized path. Alternative methods such as ADMM-based solvers (Boyd et al., 2011; Hallac et al., 2015) and split-Bregman solvers (Ye and Xie, 2011) are important for nonsmooth fused penalties, but they change the optimization architecture and introduce additional tuning and implementation choices. The comparisons here are designed to isolate the effect of preserving the sparse graph inside the existing grouped fused-regression optimization framework. The contribution of this paper is an algorithmic representation of grouped fused regression that makes the sparse graph a first-class object in the optimizer. Instead of constructing all possible group-pair fusion terms and encoding graph sparsity through zero weights, the proposed representations operate directly on the modeled graph edges. This principle yields exact active-edge L1 and

L2 reformulations, and also motivates approximate high-efficiency solvers when additional graph reduction is acceptable.

The main contributions are:

- a sparse-graph representation principle for grouped fused regression solvers;
- exact active-edge L1 and L2 reformulations that preserve the reference objective while avoiding zero-weight all-pair construction;
- complexity reductions that replace fusion-side dependence on all $\binom{k}{2}$ group pairs with dependence on the m modeled graph edges;
- a clear separation between exact sparse-graph solvers and approximate chain variants that reduce the represented graph to $k-1$ edges;
- a controlled empirical design that keeps the optimizer family fixed and isolates the effect of graph representation; and
- empirical comparisons showing that active-edge construction preserves predictive accuracy while reducing runtime and memory on public grouped-regression benchmarks.

The remainder of the paper is organized as follows. Section 2 reviews fused and graph-structured regression methods, together with related optimization approaches. Section 3 defines the grouped fused-regression model and the full-pairwise computations used as references. Section 4 presents the active-edge L1, the active-edge L2 augmented-design construction, and the chain approximation. Section 5 describes the benchmark protocol, datasets, and compared methods. Section 6 reports accuracy, timing, memory, and efficiency results. Section 7 discusses implications, limitations, and implementation guidance, and Section 8 summarizes reproducibility materials.

2 Related Work

The fused lasso extends the lasso by combining sparsity with penalties on ordered coefficient differences (Tibshirani, 1996; Tibshirani et al., 2005). Grouped fused regression adapts this idea to grouped data by estimating one coefficient vector per group and penalizing differences between coefficient vectors along a graph of group relationships. This places grouped fused regression in a broader family of structured regularizers, including the elastic net, group lasso, generalized lasso, and graph trend filtering (Zou and Hastie, 2005; Yuan and Lin, 2006; Tibshirani and Taylor, 2011; Wang et al., 2016). Specialized algorithms for one-dimensional signal approximation, including path algorithms for the fused lasso signal approximator, exploit the ordered chain structure of that problem (Hoefling, 2010). Split-Bregman and ADMM methods provide another important optimization line for nonsmooth fused penalties (Ye and Xie, 2011; Boyd et al., 2011). General graph fusion is more flexible but computationally harder because the penalty couples many coefficient blocks.

It is useful to separate three forms of fusion that are discussed under related fused-lasso terminology. In the classic fused lasso, the ordered objects being fused are usually variables, coordinates, or signal locations. For a single regression vector, this means penalizing adjacent coefficient differences such as $|\beta_{j+1} - \beta_j|$. In graph-guided multi-task learning, the

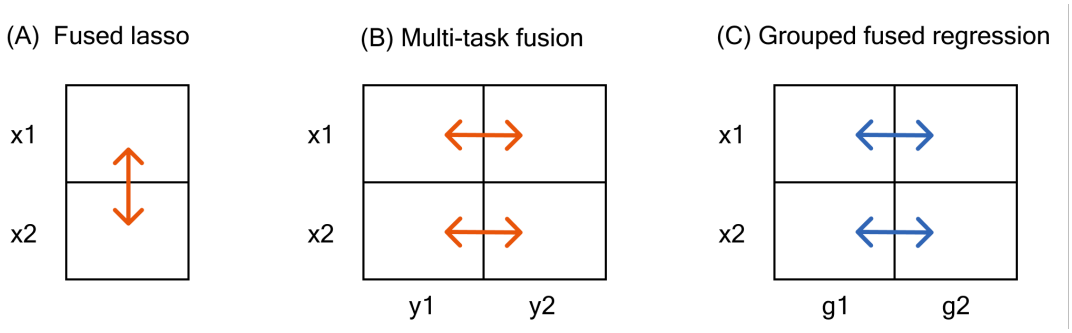

Figure 1: Three fusion forms shown as coefficient matrix schematics. Rows labeled x_1 and x_2 denote predictors. In Panel B, columns y_1 and y_2 denote related responses or tasks. In Panel C, columns g_1 and g_2 denote groups of observations. Links indicate which objects are coupled by the fusion penalty. Panel A shows fused lasso penalties on ordered coefficients (Tibshirani et al., 2005). Panel B shows graph-guided multi-task fusion across related responses or tasks (Chen et al., 2010, 2012a). Panel C shows grouped fused regression, where the fusion graph is placed on groups of observations (Dondelinger et al., 2020). This grouped fused regression, is what we focus on in this paper.

fused objects are tasks, often corresponding to related responses or prediction problems. In grouped fused regression, the fused objects are groups of observations in one regression problem. The model estimates a coefficient matrix $B \in \mathbb{R}^{p\times k}$, with one column for each group. The fusion graph is on the groups, and an edge (g, h) penalizes the difference between the two coefficient vectors B_g and B_h. The focus of this paper is entirely on fusion between groups, not fusion between variables or responses.

Figure 1 illustrates these distinctions. Classic variable fusion acts within a single ordered coefficient vector, as in the fused lasso (Tibshirani et al., 2005). Multi-task fusion encourages sharing across columns of a coefficient matrix, where the columns correspond to responses or tasks (Chen et al., 2010, 2012a). Grouped fused regression also encourages sharing across columns, but the columns are groups from one grouped regression problem with common predictors and a common response (Dondelinger et al., 2020). This is why graph-guided multi-task learning is related but not identical to the problem studied here.

Graph-structured multi-task regression extended these ideas to arbitrary task graphs and developed smoothed proximal-gradient methods for the general fused lasso (Chen et al., 2010, 2012a). Related multi-task and scientific applications use graph-guided fusion for eQTL mapping, additive modeling and gene regulatory network inference (Chen et al., 2012b; Petersen et al., 2016; Omranian et al., 2016). Network lasso methods (Hallac et al., 2015), graph trend filtering, Depth First Search (DFS) fused lasso denoising (Tansey et al., 2018), and k-nearest-neighbor fused lasso estimators (Padilla et al., 2020) further emphasize that the graph itself is part of the modeling problem. The crucial idea is that the graph may come from prior knowledge, distances between groups, or other data-driven structure.

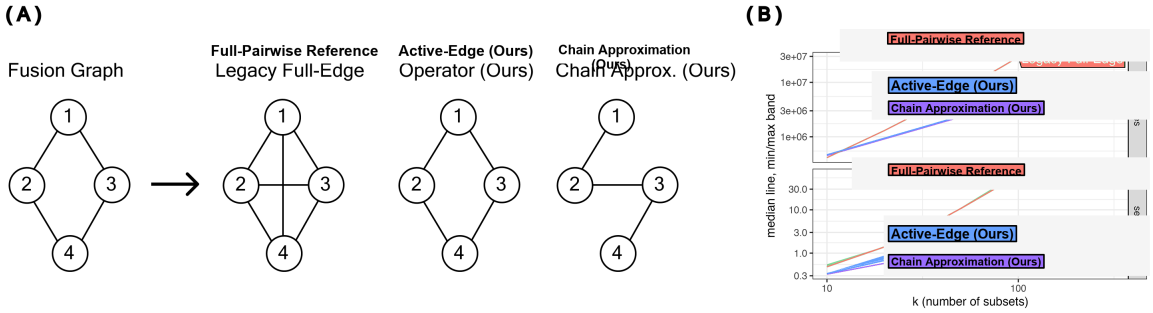

Figure 2: Conceptual overview of L1 sparse-graph fusion. A: Fusion graph construction. The reference full-edge construction associated with Dondelinger et al. (2020) forms a combined constraint matrix over all group pairs, while the active-edge operator works directly on the sparse graph. The chain approximation replaces the modeled edge set with a chain before fitting. B: Asymptotic complexity. The synthetic example uses $p = 120$ features and 35 training observations per group. Thus, when the number of groups is $k = 100$, the training set has $n = 3500$ observations. The rest of the hyper-parameters are fixed. For the sparse chain graph, where m is the number of fusion edges ($m = k - 1$), the active-edge representation changes the dominant fusion cost from $O(pk^3)$ to $O(pm)$.

The present work focuses on how grouped fused regression objectives should be represented and solved especially when the graph is sparse. The graph is part of the modeling assumption, and a dense all-pairs representation discards that assumption at the computational layer. The active-edge constructions proposed restore the intended edge-level representation while preserving the objective in the exact L1 and L2 cases.

Table 1 summarizes the existing methodological literature. The comparison is organized around modeling and optimization ideas. The novel contribution of this paper is an active-edge representation that preserves a sparse graph inside the existing grouped fused-regression objective. In this view, the closest statistical existing literatures are the joint-lasso/grouped fused-regression formulation of Dondelinger et al. (2020) and the graph-guided multitask learning and smoothed proximal-gradient methods for general fused penalties (Chen et al., 2010, 2012a). The role of the present work is to isolate the representation question. This implies that when the graph is sparse, the optimization data structures should scale with the active edge set rather than with all possible group pairs.

3 Grouped Fused Regression and Graph Computation

Figures 2 and 3 summarize the main computational distinction between the reference fused-regression representation and the sparse-graph representation. In both cases, the statistical model is defined by a graph on the k groups, with an absolute edge weight indicating how strongly two subgroup coefficient vectors should be encouraged to agree. The reference construction first expands this modeling graph into a dense algebraic object over many group pairs. That expansion is benign for small k, but it becomes the dominant cost when

Table 1: Existing methodological literature and distinctions from the present work. The paper builds on graph-fused regression and smoothed first-order optimization, but focuses on preserving sparse edge sets in the solver representation while holding the objective and optimizer family fixed.

Literature	Main idea	Distinction from this paper
Lasso and fused lasso (Tibshirani, 1996; Tibshirani et al., 2005)	Combines sparsity with penalties on adjacent coefficient differences.	Introduces the core sparsity and fusion penalties, while this paper studies how to represent sparse fusion graphs efficiently in grouped fused-regression.
Graph-guided multitask regression (Chen et al., 2010, 2012a,b)	Uses task graphs and smoothed proximal-gradient methods for general fused multitask penalties.	This paper uses related graph-fusion ideas to target grouped regression instead of general multitask learning.
Generalized lasso and graph trend filtering (Tibshirani and Taylor, 2011; Wang et al., 2016)	Uses linear operators or graph differences to encode structured variation.	Provides a broad operator framework, whereas this paper focuses on preserving sparse graph edges inside grouped fused-regression solvers.
Split-Bregman and ADMM approaches (Ye and Xie, 2011; Boyd et al., 2011)	Introduce auxiliary variables for nonsmooth penalties and solve alternating subproblems.	The comparisons in this paper keep the smoothed first-order optimization framework fixed to isolate the effect of graph representation.
Network lasso and graph-based fusion (Hallac et al., 2015)	Penalizes graph-edge differences to cluster or share information over networks.	Uses graph-edge penalties to share information over networks, but studies a different objective and optimization framework from the grouped fused-regression in this paper.
Joint lasso for subgroup regression (Dondelinger et al., 2020)	Estimates grouped regressions with sparsity and fusion across groups.	Closest statistical predecessor and reference used in this paper. This paper preserves its objective while replacing all-pair graph construction with active-edge construction.
Graph decomposition for graph-fused lasso (Yu et al., 2025)	Uses graph decomposition to build a faster ADMM algorithm for graph-fused lasso on general graphs.	Changes the ADMM decomposition and optimizer architecture; this paper keeps the smoothed L1 and augmented-design L2 solver families fixed to measure representation effects.

the intended graph is sparse, because computation is then driven by the number of possible group pairs rather than by the number of modeled edges.

Figure 2 describes the L1 fusion case. Panel A begins with the intended fusion graph. Nodes are regression groups and each drawn edge marks a coefficient difference penalized by the model. The reference full-edge construction expands that graph into an all-pairs layout for the combined L1 constraint matrix. In the four-group concept, it introduces pair slots that are absent from the modeled graph. The exact active-edge operator keeps the original graph and evaluates the nonsmooth fusion penalty only along its m nonzero edges. The chain approximation makes a different tradeoff by replacing the original edge set with a chain before fitting. Panel B then shows the computational consequence. The active-edge L1 representation avoids the all-pairs constraint matrix, so fusion-side scaling depends entirely on the graph edges rather than powers of k.

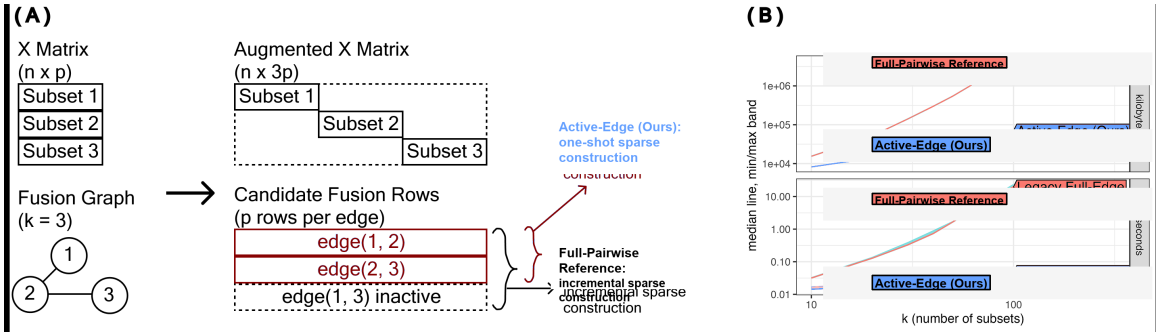

Figure 3: Conceptual overview of L2 sparse-graph fusion. A: augmented matrix construction. The reference full-edge construction associated with Dondelinger et al. (2020) forms augmented rows for all group pairs, while the active-edge builder emits only sparse triplets for graph edges with nonzero weight. B: asymptotic complexity. The synthetic example uses $p = 120$ features and 35 training observations per group, so when $k = 100$ the training set has $n = 3500$ observations. For the sparse chain graph, where m is the number of fusion edges ($m = k - 1$), active-edge construction changes the dominant fusion-block cost from $O(pk^2)$ to $O(pm)$.

Figure 3 gives the analogous picture for L2 fusion. In the quadratic case, fusion can be represented by augmenting the regression design with pseudo-observations that penalize differences between coefficient vectors assigned to connected groups. The reference construction emits augmented rows for the expanded set of group pairs. The active-edge construction emits sparse triplets only for edges with nonzero fusion weight. Thus the same modeling graph produces a much smaller linear-algebra problem whenever $m \ll k^2$. These two figures motivate the solver descriptions in this paper and the scaling experiments reported below. The central question is whether preserving the edge-level graph representation yields the same fitted objective at lower runtime and memory cost.

3.1 Optimization Guarantee for Smoothed L1

The L1 solver follows a smoothed proximal-gradient strategy for Equation 1. Let $B = [\beta_1, \ldots, \beta_k]$ be the $p \times k$ coefficient matrix. The reference implementation builds a constraint matrix with columns for sparsity and fusion constraints, then uses a smoothed dual approximation to compute the gradient contribution of the nonsmooth penalty. The update has the generic accelerated form

$$A^\star = \Pi_{[-1,1]}\left(\frac{BC}{\mu}\right), \tag{2}$$

$$\nabla f_\mu(B) = \nabla \ell(B) + A^\star C^\top, \tag{3}$$

$$B^{t+1} = W^t - L^{-1}\nabla f_\mu(B^t), \tag{4}$$

where C encodes the sparsity and fusion operators, μ is the smoothing parameter, L is a Lipschitz bound, and W^t is the accelerated iterate. When C contains all group pairs,

storage and multiplication include $O(pk^2)$ fusion terms for a complete representation. More explicitly, the L1 implementation uses the dual identity $|z| = \max_{|a|\le 1} az$ and replaces the non-smooth absolute value terms by clipped smoothed duals. For sparsity and active fusion edges this gives

$$A_s^\star = \operatorname{clip}\left(\frac{\lambda B_s}{\mu}, -1, 1\right), \qquad A_{uv}^\star = \operatorname{clip}\left(\frac{\gamma w_{uv}(\beta_u - \beta_v)}{\mu}, -1, 1\right),$$

where B_s excludes intercept rows. The code uses the step-size upper bound

$$L_U = \lambda_{\max} + \frac{\lambda^2 + 2\gamma^2 d_{\max}^{(2)}}{\mu},$$

with $\lambda_{\max} = \sum_g \lambda_{\max}(X_g^\top X_g/n_g)$ under scaled group losses and $d_{\max}^{(2)}$ any upper bound on $\max_g \sum_{h:(g,h)\in E} w_{gh}^2$ for the active fusion graph. This quantity is bounded by the usual weighted degree when weights are normalized to lie in $[0, 1]$. The guarantees below are deterministic statements for a fixed graph and fixed hyper-parameters. They concern objective representation and optimization, not statistical recovery under a data generating model. The first guarantee specializes the standard consequences of Nesterov smoothing and accelerated proximal-gradient convergence to the grouped fused-regression notation used here (Nesterov, 2005; Beck and Teboulle, 2009).

Proposition 1 (Lipschitz and convergence guarantee for smoothed L1)
Let $F_\mu(B)$ be the Nesterov-smoothed L1 grouped fused objective, before final postprocessing. Let $B_\mu^\star$ minimize F_μ. If the Accelerated Proximal Gradient (APG) update uses any L_U satisfying

$$L_U \ge \lambda_{\max} + \frac{\lambda^2 + 2\gamma^2 d_{\max}^{(2)}}{\mu},$$

then ∇F_μ is L_U-Lipschitz and the accelerated iterates satisfy

$$F_\mu(B^{(t)}) - F_\mu(B_\mu^\star) \le \frac{2L_U \left\|B^{(0)} - B_\mu^\star\right\|_F^2}{(t+1)^2}.$$

Moreover, if N_c is the number of scalar sparsity and fusion constraints in the active-edge representation, then the nonsmoothed objective F and its smoothed version obey

$$0 \le F(B) - F_\mu(B) \le \frac{\mu N_c}{2}$$

for every coefficient matrix B.

Proof The least-squares part has Lipschitz constant bounded by $\lambda_{\max}$ under the scaled group losses. The smoothed sparsity operator contributes at most λ^2/μ to the gradient Lipschitz constant. For the fusion operator, each edge difference has two incident group blocks. The squared operator norm is bounded by $2d_{\max}^{(2)}$, giving the fusion contribution $2\gamma^2 d_{\max}^{(2)}/\mu$. Summing these bounds gives L_U. The stated $O(t^{-2})$ rate is the standard

accelerated-gradient guarantee for convex differentiable objectives with L_U-Lipschitz gradient. Finally, each scalar absolute-value term satisfies $0 \le |z| - \max_{|a|\le 1}\{az - \mu a^2/2\} \le \mu/2$; summing over the N_c scalar constraints gives the smoothing gap. ■

Proposition 2 (Equivalence of dense and active-edge L1 gradients)

Fix a weighted graph G and let E denote the edges with positive weight. For smoothing parameter $\mu > 0$, suppose the dense smoothed proximal-gradient implementation includes one fusion column for every pair but assigns zero weight to inactive pairs. The active-edge operator implementation computes exactly the same smoothed fusion gradient as the dense implementation after removing the zero-weight columns.

Proof Let e_u be the uth coordinate vector in $\mathbb{R}^k$. For any pair $1 \le u < v \le k$, we define the smoothed dual block

$$P_{uv}(B) = \operatorname{clip}\left(\frac{\gamma w_{uv}(\beta_u - \beta_v)}{\mu}, -1, 1\right).$$

The dense fusion contribution to the smoothed gradient can be written as

$$\Delta_{\text{dense}}(B) = \sum_{1\le u<v\le k} \gamma w_{uv} P_{uv}(B)(e_u - e_v)^\top.$$

If $w_{uv} = 0$, then the corresponding summand is zero. Therefore

$$\Delta_{\text{dense}}(B) = \sum_{(u,v)\in E} \gamma w_{uv} P_{uv}(B)(e_u - e_v)^\top = \Delta_{\text{active}}(B).$$

The active-edge APG algorithm computes this last expression by adding $\gamma w_{uv} P_{uv}(B)$ to column u and subtracting it from column v for each active edge. Hence the dense and active-edge implementations produce the same smoothed fusion-gradient contribution, and therefore the same L1 update when the remaining APG quantities are held fixed. ■

Algorithm 1 Full-Pairwise APG Reference

Require: grouped data, G, $\lambda, \gamma, \mu, T, \varepsilon$

Construct full fusion matrix $C_{\text{fusion}} \in \mathbb{R}^{k \times \binom{k}{2}}$
Construct $C = [\lambda I_k, \gamma C_{\text{fusion}}]$
Initialize $B^{(0)} = 0,\ W^{(0)} = 0,\ S^{(0)} = 0$
for $t = 1, \ldots, T$ **do**
    Compute $\nabla \ell(B^{(t-1)})$ groupwise
    $A^\star \leftarrow \text{clip}\Big([B_s^{(t-1)} C_s,\ B_f^{(t-1)} C_f]/\mu, -1, 1\Big)$
    $\nabla r_\mu(B^{(t-1)}) \leftarrow A^\star C^\top$
    $H_t \leftarrow \nabla \ell(B^{(t-1)}) + \nabla r_\mu(B^{(t-1)})$
    Compute L_U from the dense max-degree proxy
    Update $\tilde{B}^{(t)}, S^{(t)}, W^{(t)}$
    if $\left\| \tilde{B}^{(t)} - B^{(t-1)} \right\|_1 < \varepsilon p$ **then**
        break
    end if
    $B^{(t)} \leftarrow \tilde{B}^{(t)}$
end for
Soft-threshold non-intercept rows by λ / n_g
return $\hat{B}$

3.2 Full-Edge L2 Augmented Design

The L2 solver uses a different reduction. It constructs an augmented response and design matrix,

$$\tilde{y} = \begin{bmatrix} y \\ 0 \end{bmatrix}, \quad \tilde{X} = \begin{bmatrix} X_{\text{block}} \\ X_{\text{fusion}} \end{bmatrix},$$

where X_{block} is block diagonal over groups and X_{fusion} contains, for each group pair, rows proportional to

$$\sqrt{\gamma w_{gh}}(\beta_g - \beta_h).$$

The resulting lasso problem is passed to `glmnet`. A full all-pairs construction creates $O(pk^2)$ fusion rows even when only $m = |E|$ edges are active.

Proposition 3 (Equivalence of active-edge L2 augmented rows) *For a fixed graph G, penalty γ, and scaling convention, the active-edge L2 augmented design is equivalent to the all-pairs augmented design after removing all zero-weight pair rows and applying a row permutation. The active-edge L2 design gives the same optimization problem as the all-pairs design, except it omits rows that were zero anyway.*

Proof Each nonzero edge (u, v) contributes p_{eff} augmented rows with $+\sqrt{\gamma w_{uv}}$ in coefficient block u and $-\sqrt{\gamma w_{uv}}$ in coefficient block v, with zero augmented response. These rows contribute

$$\gamma w_{uv} \left\| \beta_u - \beta_v \right\|_2^2$$

to the augmented least-squares objective. Therefore the all-pairs fusion contribution satisfies

$$\sum_{1\le u<v\le k} \gamma w_{uv} \left\|\beta_u - \beta_v\right\|_2^2 = \sum_{(u,v)\in E} \gamma w_{uv} \left\|\beta_u - \beta_v\right\|_2^2,$$

because inactive pairs have $w_{uv} = 0$. The active-edge builder constructs exactly the rows corresponding to the right-hand side using sparse triplets. Permuting those rows changes only their order, not the sum of squared residuals, so the objective minimized by `glmnet` is unchanged. ■

Proposition 4 (Sparse-graph construction cost)
Let k be the number of groups, p the number of penalized coefficients per group, $m = |E|$ the number of nonzero fusion edges, and $q = \binom{k}{2}$ the number of all possible group pairs. In one L1 smoothed-gradient evaluation, a dense all-pairs operator performs fusion work proportional to pq after construction, whereas the active-edge operator performs fusion work proportional to pm. In the L2 augmented-design construction, the fusion block decreases from pq augmented rows to pm augmented rows.

Proof For L1 fusion, each represented edge requires a p-dimensional coefficient difference and a p-dimensional scatter back to the incident groups. Thus, the fusion-side work scales as

$$W_{\text{L1,full}} \asymp pq = p\binom{k}{2}, \qquad W_{\text{L1,active}} \asymp pm.$$

The dense all-pairs representation iterates over q candidate pairs, including zero-weight inactive pairs, while the active-edge representation iterates only over the m nonzero graph edges.

For L2 fusion, each represented edge contributes p_{eff} augmented rows, one for each penalized coefficient coordinate. Therefore the fusion block sizes are

$$R_{\text{L2,full}} = p_{\text{eff}} q = p_{\text{eff}} \binom{k}{2}, \qquad R_{\text{L2,active}} = p_{\text{eff}} m.$$

Deleting zero-weight pairs gives the stated row reduction, with p_{eff} determined by the intercept convention. ■

4 Sparse-Graph Solvers

4.1 Exact L1 Operator Solver

Here $B = [\beta_1, \ldots, \beta_k]$ is the grouped coefficient matrix, with one column per group. The exact L1 operator solver replaces the dense fusion matrix representation with edge lists $(u_e, v_e, w_e)_{e=1}^m$, where $m = |E|$ is the number of active fusion edges, u_e and v_e are the incident groups for edge e, and $w_e > 0$ is its fusion weight. At each iteration, the solver computes the fusion differences only on active edges,

$$D_e = \beta_{u_e} - \beta_{v_e}, \quad P_e = \text{clip}\left(\frac{\gamma w_e D_e}{\mu}, -1, 1\right),$$

where D_e is the coefficient difference vector and P_e is its clipped smoothed dual block. The solver scatters $\gamma w_e P_e$ back to the incident group columns with opposite signs. The sparsity part of the update remains unchanged. This representation has the same objective and first-order updates as the active-edge version of the smoothed proximal-gradient solver, but the fusion-gradient work scales with pm rather than pk^2.

The implementation also supports block processing of edges. This is a practical memory safeguard for large p or m, since intermediate edge-difference matrices can otherwise dominate the working set. The operator solver is therefore intended as the general exact path for sparse graphs.

Active-edge gradient assembly. The implementation forms the fusion-gradient contribution by initializing $\Delta_f = 0$ and looping over active edges $e = 1, \ldots, m$. For each edge, it computes the per-edge difference $d_e = \beta_{u_e} - \beta_{v_e}$ and clipped dual block $p_e = \mathrm{clip}((\gamma w_e/\mu)d_e, -1, 1)$, then applies the scatter updates

$$\Delta_f(:, u_e) \leftarrow \Delta_f(:, u_e) + \gamma w_e p_e, \qquad \Delta_f(:, v_e) \leftarrow \Delta_f(:, v_e) - \gamma w_e p_e.$$

The accumulated matrix Δ_f is then added to the likelihood and sparsity gradient terms in the accelerated update.

Algorithm. Algorithm 2 gives the active-edge APG procedure used in the experiments. Here, T is the maximum number of APG iterations and ε is the stopping tolerance.

Algorithm 2 Active-Edge Operator APG

Require: grouped data, edge list E, $\lambda, \gamma, \mu, T, \varepsilon$
Initialize $B^{(0)} = 0$, $W^{(0)} = 0$, $S^{(0)} = 0$
for $t = 1, \ldots, T$ **do**
  Compute groupwise likelihood gradient $\nabla \ell(B^{(t-1)})$
  $A_s^\star \leftarrow \mathrm{clip}(\lambda B_s^{(t-1)}/\mu, -1, 1)$
  Initialize fusion accumulator $\Delta_f \leftarrow 0$
  for each edge $e = (u_e, v_e, w_e) \in E$ **do**
    $d_e \leftarrow \beta_{u_e}^{(t-1)} - \beta_{v_e}^{(t-1)}$
    $p_e \leftarrow \mathrm{clip}((\gamma w_e/\mu)d_e, -1, 1)$
    $\Delta_f(:, u_e) \leftarrow \Delta_f(:, u_e) + \gamma w_e p_e$
    $\Delta_f(:, v_e) \leftarrow \Delta_f(:, v_e) - \gamma w_e p_e$
  end for
  $H_t \leftarrow \nabla \ell(B^{(t-1)}) + \lambda A_s^\star + \Delta_f$
  Compute L_U on active graph; update $\tilde{B}^{(t)}, S^{(t)}, W^{(t)}$
  if $\left\| \tilde{B}^{(t)} - B^{(t-1)} \right\|_1 < \varepsilon p$ **then**
    break
  end if
  $B^{(t)} \leftarrow \tilde{B}^{(t)}$
end for
Soft-threshold non-intercept rows by λ/n_g
return $\hat{B}$

4.2 Chain Approximation

The chain approximation solver builds a depth-first traversal of a maximum spanning tree or of the thresholded graph, reorders the groups along that traversal, and keeps only the $k-1$ adjacent chain edges. The fusion operator can then be computed with vectorized adjacent differences:

$$D_j = \beta_j - \beta_{j+1}, \quad j = 1, \ldots, k-1.$$

The vectorized implementation forms

$$D = B_{:,1:(k-1)} - B_{:,2:k}, \qquad P = \mathrm{clip}((\gamma/\mu) D\,\mathrm{diag}(w), -1, 1),$$

and accumulates $\gamma P\,\mathrm{diag}(w)$ on neighboring chain endpoints. This reduces the edge budget to the minimum needed to connect the groups. It does not optimize the same objective as the original graph unless the graph is already the selected chain. It is useful when the main objective is maximum speed and memory reduction and the user is willing to accept a small loss in graph fidelity.

Proposition 5 (Chain approximation target)
Let $G_\pi = (V, E_\pi)$ be the chain graph produced by the DFS/MST ordering, with edge weights inherited from the original graph. The chain approximation solver applies the exact active-edge L1 update to the smoothed fused objective on G_π. Define the original graph-fusion penalty as

$$R_G(B) = \gamma \sum_{(u,v)\in E} w_{uv} \left\|\beta_u - \beta_v\right\|_1,$$

and define $R_{G_\pi}(B)$ analogously on the chain. Then, for any coefficient matrix B and any chain with $E_\pi \subseteq E$,

$$0 \le R_G(B) - R_{G_\pi}(B) = \gamma \sum_{(u,v)\in E\setminus E_\pi} w_{uv} \left\|\beta_u - \beta_v\right\|_1.$$

Thus, the approximation error is exactly the omitted-edge fusion penalty for the fitted coefficient matrix.

Proof After the groups are reordered by π, the vectorized adjacent-difference operator is the active-edge operator for the chain edges (π_j, π_{j+1}), $j = 1, \ldots, k-1$. The scatter updates therefore match the L1 active-edge update on G_π. Moreover,

$$R_G(B) - R_{G_\pi}(B) = \gamma \sum_{(u,v)\in E} w_{uv} \left\|\beta_u - \beta_v\right\|_1 - \gamma \sum_{(u,v)\in E_\pi} w_{uv} \left\|\beta_u - \beta_v\right\|_1.$$

If $E_\pi \subseteq E$, the second sum removes exactly the retained chain edges, leaving

$$R_G(B) - R_{G_\pi}(B) = \gamma \sum_{(u,v)\in E\setminus E_\pi} w_{uv} \left\|\beta_u - \beta_v\right\|_1 \ge 0.$$

■

Optimization algorithm. The chain approximation algorithm first builds the DFS/MST order and adjacent chain weights, then runs a separate vectorized APG routine rather than calling the general edge loop. In Algorithm 3, $\eta = 1/L_U$ is the step size, H_t is the smoothed gradient at iteration t, $R^{(t)}$ is the accumulated scaled-gradient term, $Z^{(t)} = -R^{(t)}$ is the aggregate point used in the acceleration step, and $W^{(t)}$ is the accelerated iterate.

Algorithm 3 Chain Approximation APG

Require: grouped data, G, chain settings, $\lambda, \gamma, \mu, T, \varepsilon$

Build DFS/MST chain order π and adjacent chain weights $(w_1, \ldots, w_{k-1})$
Reorder groups by π; initialize $B^{(0)} = 0,\ W^{(0)} = 0,\ R^{(0)} = 0$
Compute chain Lipschitz bound L_U and set $\eta \leftarrow 1/L_U$
for $t = 1, \ldots, T$ **do**
    Compute groupwise likelihood gradient $\nabla \ell(B^{(t-1)})$
    $A_s^\star \leftarrow \mathrm{clip}(\lambda B_s^{(t-1)}/\mu, -1, 1)$
    $\Delta_f(:, 1:(k-1)) \mathrel{+}= C, \qquad \Delta_f(:, 2:k) \mathrel{-}= C$
    $H_t \leftarrow \nabla \ell(B^{(t-1)}) + \lambda A_s^\star + \Delta_f$
    $\tilde{B}^{(t)} \leftarrow W^{(t-1)} - \eta H_t$
    $R^{(t)} \leftarrow R^{(t-1)} + \frac{t\eta}{2} H_t$
    $Z^{(t)} \leftarrow -R^{(t)}$
    $W^{(t)} \leftarrow \left(t\tilde{B}^{(t)} + 2Z^{(t)} \right) / (t+2)$
    if $\left\| \tilde{B}^{(t)} - B^{(t-1)} \right\|_1 < \varepsilon p$ **then**
        break
    end if
    $B^{(t)} \leftarrow \tilde{B}^{(t)}$
end for
Soft-threshold non-intercept rows by λ / n_g
Reorder columns back to original group order
return $\hat{B}$

4.3 Active-Edge L2 Augmented Design

The L2 active-edge solver preserves the augmented-design strategy but changes how the design is built. Instead of allocating all pairwise fusion blocks and then assigning zero-weight rows, it finds the upper-triangular nonzero entries of G and builds sparse triplets only for those edges. The p_{eff} denotes the number of coefficient coordinates included in the L2 fusion block, and $n_{\text{fusion}} = p_{\text{eff}} m$ is the number of active fusion rows.

The active-edge construction keeps the same augmented-design scaling:

$$\sqrt{\gamma(n + n_{\text{fusion}})}\sqrt{w_{gh}}$$

multiplies the positive and negative entries in the fusion rows. The final augmented design is passed to `glmnet` (Friedman et al., 2010) with the lambda correction factor. This change leaves the fitted objective unchanged for a given active graph but reduces the number of constructed fusion rows from $p_{\text{eff}} k(k-1)/2$ to $p_{\text{eff}} m$. The largest expected gains occur when $m \ll k(k-1)/2$.

Algorithm 4 L2 Augmented Design Solver with `glmnet`

Require: grouped data (X, y, groups), graph G, λ, γ, intercept and loss-scaling conventions

Build block-diagonal design X_{block} over groups
Extract active edges $E = \{(u, v) : u < v,\ G_{uv} \neq 0\}$
Let $\mathcal{P}$ be the coefficient coordinates included in L2 fusion, with $|\mathcal{P}| = p_{\text{eff}}$
Initialize sparse triplet lists for X_{fusion} and set row counter $a \leftarrow 0$
for each active edge $(u, v) \in E$ **do**
    for each coefficient coordinate $r \in \mathcal{P}$ **do**
        $a \leftarrow a + 1$
        Add triplet $(a, \text{col}(u, r), +\sqrt{w_{uv}})$
        Add triplet $(a, \text{col}(v, r), -\sqrt{w_{uv}})$
    end for
end for
Set $n_{\text{fusion}} \leftarrow a$ and materialize sparse matrix X_{fusion} from the triplets
Scale fusion block by $\sqrt{\gamma(n + n_{\text{fusion}})}$
Form augmented system $\tilde{X} = [X_{\text{block}}; X_{\text{fusion}}]$, $\tilde{y} = [y; 0]$
Fit θ with `glmnet` on $(\tilde{X}, \tilde{y})$ using λ
Reshape θ to $B \in \mathbb{R}^{p_{\text{eff}} \times k}$
return B

Optimization algorithm. Algorithm 4 summarizes the L2 path, which uses `glmnet` (Friedman et al., 2010) after building an augmented sparse regression problem. The fitted vector θ stacks the groupwise coefficients and is reshaped into the coefficient matrix B after fitting:

Table 2 summarizes the asymptotic costs implied by the preceding solver descriptions. The purpose is to separate the common groupwise regression cost, represented by $O(np)$ terms, from the cost of the fusion representation. The L1 time entries describe one APG iteration, while the L2 time entries describe one pass over the augmented design. For L2, the table uses p_{eff} because the intercept convention determines how many coefficient coordinates are included in the fusion block. The active-edge L1 memory bound assumes the block processing, so edge-difference workspaces are not materialized for all m edges at once.

5 Experimental Design

5.1 Empirical Test Problems

The real data experiments use public grouped regression problems as optimization test cases. They provide different values of n, p, k, and graph sparsity while keeping the statistical objective fixed across solver representations. Across these datasets, the number of observations ranges from 630 to 200,000, the number of features from 3 to 100, and the number of groups from 43 to 229. Most of the real data test problems are drawn from Rdatasets mirrors (Arel-Bundock, 2026). The remaining sources are UCI datasets (Redmond, 2002; Trindade, 2015), Our World in Data CO2 data (Ritchie et al., 2023), World Bank indica-

Table 2: Dominant representation costs of the solver methods. Here n is total sample size, p is feature count, p_{eff} is the number of coordinates included in the L2 fusion block, k is group count, $m = |E|$ is the number of active fusion edges, and $q = \binom{k}{2}$. The table isolates graph-representation costs and omits lower-order constants from solver overhead.

Method	Time	Memory
L1 Full-Pairwise Reference	$O(np + pkq)$	$O(np + pk + pq + kq)$
L1 Active-Edge	$O(np + pm)$	$O(np + pk + m)$
L1 Chain Approximation	$O(np + pk)$	$O(np + pk)$
L2 Full-Pairwise Reference	$O(np + p_{\text{eff}}q)$	$O(np + p_{\text{eff}}q)$
L2 Active-Edge	$O(np + p_{\text{eff}}m)$	$O(np + p_{\text{eff}}m)$

tors (World Bank, 2026), and package datasets (Pinheiro and Bates, 2000; Raudenbush and Bryk, 2002).

5.2 Evaluation Protocol

All reported real data results use grouped outer cross-validation with five stratified folds (Kohavi, 1995; Varma and Simon, 2006). Splits are made at the observation level while preserving group representation where possible. All graph construction, hyperparameter selection, and standardization are performed inside the outer-training fold so that the held-out test fold is not used in model selection.

For each outer fold, a baseline sparsity penalty is selected with fusion disabled, $\gamma = 0$, on the outer-training data by `cv.glmnet` with three inner folds and `lambda.min`. We denote this selected sparsity penalty by λ in Equation 1. Holding this λ fixed, a separate inner train-validation split of the outer-training data selects the fusion penalty γ in Equation 1 by validation root mean squared error (RMSE). The goal of the experiments is to compare solver representations under a common protocol, not to optimize hyperparameter-search efficiency. It was sufficient to select nontrivial fused models that outperform the featureless baseline in the reported result panels. The final model is then refit on the full outer-training fold using the selected λ and γ and evaluated on the held-out outer fold. The featureless baseline predicts the mean of the training fold for the corresponding response. Timing measures only the model-fitting step, and memory is the maximum RAM used by the fitting process.

We ran the computational experiments on Northern Arizona University's HPC infrastructure (Monsoon), an x86_64 Linux cluster running Red Hat Enterprise Linux 8 and scheduled with Slurm (Northern Arizona University Advanced Research Computing, 2026). The experiments and analysis code were written in R and run with R version 4.5.2. The implementation uses R and Rcpp/C++, with compiled code built through R's standard package toolchain and linear algebra calls handled by the BLAS and LAPACK libraries available through the R installation. Timing experiments were submitted as Slurm array jobs with one CPU core per task and a 24-hour time limit. Jobs were split into small,

medium, and large memory tiers and requested 16 GB, 32 GB, and 64 GB of memory, respectively. The tier assignment used a deterministic workload score based on n, p, and k of the data. Each job ran one dataset experiment on a single compute node under the same scheduling protocol. The job scripts do not request exclusive allocation or pin jobs to a particular Monsoon node generation, so absolute wall-clock times may reflect normal shared-cluster and hardware variability. For this reason, timing results are interpreted primarily as relative comparisons between solver representations run under the same scheduling protocol.

5.3 Graph Construction and Hyperparameters

The real data benchmark constructs a data-driven group graph from training data, following the use of graph-based relationships and k-nearest-neighbor fusion graphs in prior work (Hallac et al., 2015; Padilla et al., 2020). Group centroids are computed on the outer-training fold, a k-nearest-neighbor graph is formed from centroid distances, inverse-distance weights are used, and connectivity is enforced by adding nearest inter-component links. Graphs are constructed using $k = 5$ nearest neighbors (Hallac et al., 2015; Madrid Padilla et al., 2020), inverse-distance edge weights (Hallac et al., 2015), and minimum-spanning-tree augmentation to ensure connectivity.

This follows the general idea that graph structure should encode similarity between groups rather than defaulting to a complete graph. A complete graph may still be appropriate when every group pair is intended to be fused, but in that case the active-edge representation has $m = \binom{k}{2}$ and therefore does not provide the sparse-graph efficiency gains targeted in this paper. Each dataset is evaluated with five outer folds. The regularization parameter λ is warm-started from `cv.glmnet`; the fusion parameter γ is selected from the grid 10^{-6} to 10^{-1}. The L1 optimization tolerance is 10^{-5} with a maximum of 1200 iterations. The L2 panel uses the same outer-fold and graph protocol with the L2 solvers.

5.4 Compared Methods

For L1 fusion, we compare the reference full construction, the exact active-edge operator, the chain approximation, and a featureless baseline. For L2 fusion, we compare the reference augmented design, the active-edge augmented design, and the featureless baseline.

The exact comparisons are intentionally conservative. The operator and active-edge L2 methods are compared against reference implementations using the same graph, outer folds, selected hyperparameters, and prediction pipeline. Thus the reported differences isolate representation and construction costs, not changes in the statistical model or optimizer family. Comparisons against ADMM (Boyd et al., 2011; Hallac et al., 2015), split-Bregman (Ye and Xie, 2011), or other edge-splitting optimizers answer a broader question about optimizer choice. The present experiments instead test whether the sparse graph can be preserved while keeping the L1 solver within the smoothed proximal-gradient family (Chen et al., 2012a) and the L2 solver within the augmented-design family. The chain-approximation L1 method is reported separately because it changes the graph and is therefore an approximation rather than an exact replacement.

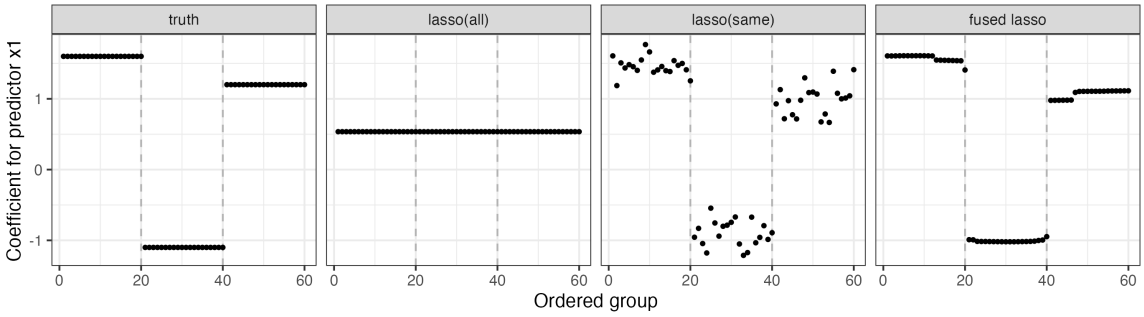

Figure 4: Controlled changepoint example comparing fused regression with unfused lasso baselines. Points show the coefficient for the first predictor across all 60 ordered groups, and dashed vertical lines mark the true changepoints. lasso(all) fits only one shared coefficient path, lasso(same) produces noisy separate paths, and the fused lasso model recovers the piecewise-constant coefficient structure. The lasso baselines are fit with `glmnet`.

6 Results

The first experiment illustrates when graph fusion is statistically useful relative to unfused `glmnet` baselines. The remaining results evaluate the main optimization question of the paper, namely whether sparse-graph representations preserve predictive accuracy while reducing runtime and memory for L1 and L2 grouped fused regression.

6.1 Changepoint Structure and the Need for Fusion

Before comparing solver representations, we first show a case where fusion is statistically useful relative to unfused lasso baselines. Unfused lasso regression implementations such as `glmnet` are highly optimized and computationally efficient. When the application does not require fusion across groups, these baselines are certainly preferable to fused-regression alternatives. The purpose of this experiment is to clarify why a fused lasso regression model is even useful in the first place. The data generation process has ordered groups whose regression coefficients are neither globally identical nor unrelated across groups. The intended structure is piecewise constant across three contiguous blocks: groups 1–20, groups 21–40, and groups 41–60 each have stable coefficients, with changes only at the two boundaries between these blocks. We generated an ordered-group regression problem with $k = 60$ groups and $p = 20$ predictors. The true coefficient paths are piecewise constant: groups 1–20 share one coefficient block, groups 21–40 share a second block, and groups 41–60 share a third block. Therefore, the two true changepoints occur after groups 20 and 40. Each group has 25 training observations for fitting, 10 validation observations for tuning, and 80 test observations. The fusion graph is the chain over the ordered groups, so the fused model penalizes differences only between neighboring groups.

We compare three estimators. The lasso(all) baseline fits one shared coefficient vector using all groups together, so it is stable but cannot represent changepoints for each group.

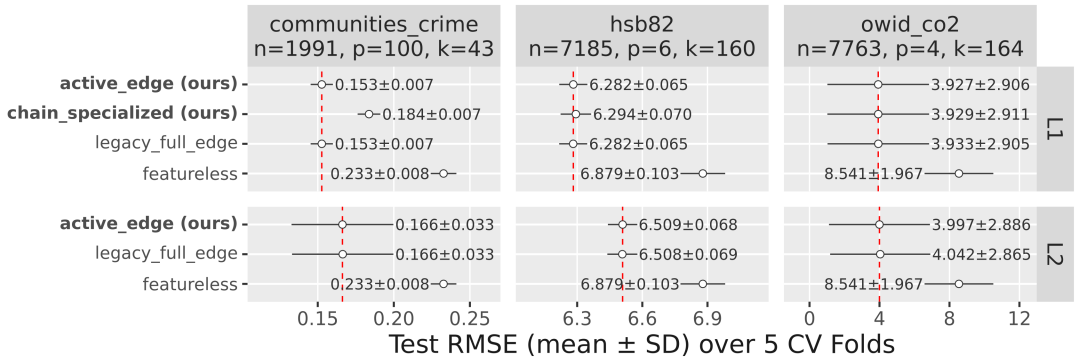

Figure 5: Prediction accuracy for L1 and L2 fusion on real grouped regression datasets. Points show test RMSE means over five cross-validation folds, with horizontal bars showing the standard deviation. The vertical red lines indicate the full pairwise baseline error which we want to preserve with the other methods. Exact active-edge solvers preserve the full-construction RMSE, while the chain approximation is an approximate method.

The lasso(same) baseline uses interaction terms to fit separate coefficient vectors for each group without any fusion penalty. It does not borrow strength across neighboring groups. The fused lasso model also estimates coefficient vectors for each group, but adds an L1 fusion penalty on adjacent groups in the chain so that it balances shared signal against differences among groups. Both lasso baselines are fit with `glmnet`. Their lasso penalties are selected by five-fold `cv.glmnet` on the training data, while the fused lasso selects (λ, γ) by validation RMSE over a fixed grid.

Figure 4 shows the fitted path for the first predictor across the 60 ordered groups. The lasso(all) path is nearly flat because it must use the same coefficient vector for all groups. The lasso(same) path varies by group, but without fusion it produces a noisy path with many spurious jumps. The fused lasso path is piecewise constant and changes at the two true changepoints.

Numerically, the fused lasso obtains the lowest test RMSE (0.708), the lowest relative coefficient error (0.079), and exactly two large estimated jumps, both at the true changepoints. By comparison, lasso(all) has test RMSE 1.957 and relative coefficient error 0.848, while lasso(same) improves prediction error but produces 59 large jumps.

6.2 Accuracy Preservation

Figure 5 compares prediction accuracy for L1 and L2 fusion on three real grouped regression datasets. These datasets span a high-feature setting with fewer groups (`communities_crime`) and two higher group settings with moderate feature counts (`hsb82` and `owid_co2`). The figure is intended to answer the research question, does replacing all-pair construction with active-edge construction change the test error? For the exact sparse-graph solvers, we could see that our optimization approaches does not change the accuracy. The L1 active-edge op-

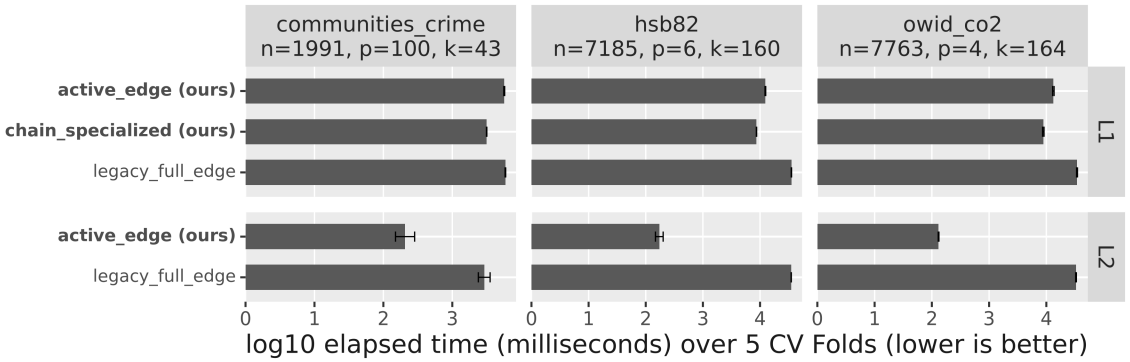

Figure 6: Fit time for the same datasets and solver families shown in Figure 5. Bars show mean $\log_{10}$ elapsed time in milliseconds over five cross-validation folds, with error bars showing the standard deviation. Lower values are better.

erator overlaps the full-edge L1 reference on all three datasets, and the L2 active-edge augmented design similarly overlaps the full-pairwise L2 reference. This matches the equivalence results in Section 4. The active-edge representation changes the data structure used by the solver, not the fitted objective. The chain-approximation L1 method is shown separately in the L1 row because it is an approximation; its accuracy can differ because replacing the graph by a chain changes the fusion penalty.

6.3 Timing Gains

Figure 6 reports the corresponding fit times for the same three datasets and solver families. The timing figure addresses the second part of the empirical claim. Once accuracy is preserved, the benefit of the active-edge representation is computational. The L1 operator avoids applying fusion updates to inactive graph pairs, while the L2 builder avoids materializing zero-weight augmented rows before fitting the model.

The number of groups is the main factor of this efficiency gap. A full pairwise construction has $q = \binom{k}{2}$ candidate group pairs, so increasing k increases the graph side of the computation quadratically even when the intended graph is sparse. In contrast, the active-edge representation scales with the number of nonzero graph edges m. For sparse graphs such as nearest-neighbor or chain graphs, m grows much more slowly than $\binom{k}{2}$; for a chain, $m = k - 1$. This is why the high-k datasets `hsb82` and `owid_co2` show the clearest timing separation.

The L2 timing gains are especially large because all-pair augmented-design construction is expensive when k is large and the graph is sparse. The L1 operator also improves timing, although the gain is smaller because the smoothed proximal-gradient iterations retain fixed overhead beyond graph updates. The chain-approximation method is fastest among the L1 methods in these examples, but it is interpreted as an accuracy-efficiency tradeoff instead of an exact replacement for the sparse graph.

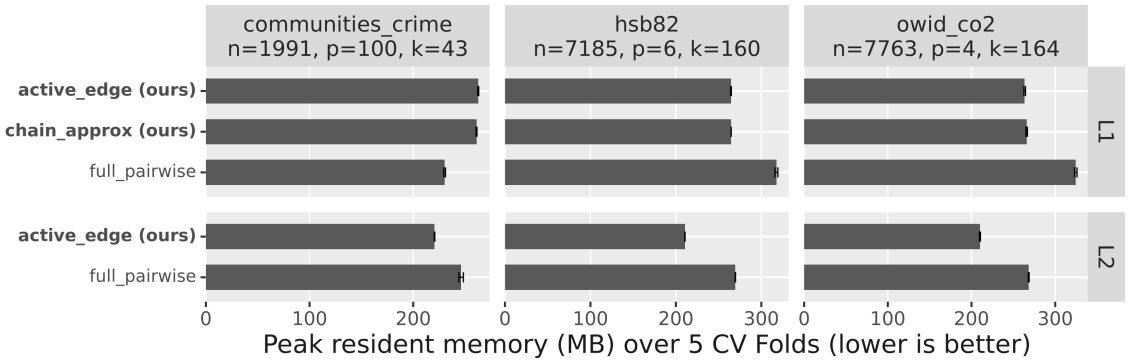

Figure 7: Peak resident memory for the same representative datasets and solver families shown in Figure 5. Bars show mean peak resident set size in MB over five cross-validation folds, with error bars showing one standard deviation. Lower values are better.

6.4 Memory Gains

Figure 7 reports the corresponding absolute peak resident memory. The figure should therefore be read as a memory comparison in a process, not as a direct measurement of the graph object alone. This helps in interpreting the optimization results. For instance, sparse construction reduces the graph objects created by the solver, but peak resident memory also includes the design matrix, response vector, group indices, coefficient iterates, validation state, and runtime allocations.

The L1 panel shows why memory is a weaker and less uniform outcome than timing. For the lower-k `communities_crime` problem, the active-edge and chain fits use slightly more peak memory than the full-pairwise reference, indicating that fixed data objects and temporary APG allocations dominate the peak RSS. For the higher-k `hsb82` and `owid_co2` problems, the active-edge and chain fits use less memory than the full-pairwise reference. This is the expected pattern when the number of candidate pairs $\binom{k}{2}$ becomes large relative to the sparse edge set. The chain approximation further reduces the graph object, although it changes the fusion penalty.

The L2 panel gives a cleaner memory story that the active-edge augmented design uses less peak memory than the full-pairwise reference in all three datasets. This follows from the construction itself. The full-pairwise reference can create augmented rows for all candidate group pairs before zero-weight pairs are discarded, whereas the active-edge builder creates rows only for nonzero graph edges. The resulting sparse regression problem contains fewer rows, sparse-matrix entries, row indices, and construction triplets before calling `glmnet`. The reduction is visible for all three datasets and is epsecially largest for the higher-k examples.

6.5 Runtime-Memory Efficiency Gains

Figure 8 summarizes the exact L1 operator gains across the real data benchmark panel. Each point compares the active-edge L1 operator with the full-edge L1 reference on the same dataset. The horizontal coordinate is the log ratio of fit time, and the vertical coordinate is the log ratio of peak memory. Points to the left of zero are faster than the full-edge reference, points below zero use less memory, and points in the lower-left quadrant improve both. Labels report the dataset, the number of groups, and the percent RMSE change relative to the full-edge reference.

The plot is interpreted as a standard multiobjective efficiency comparison, where methods are compared by accuracy and computational cost. A method is preferable only if it improves one resource without worsening the other, or if the tradeoff is explicit. This is relevant because sparse-graph solvers are intended to reduce both arithmetic work and the amount of graph-derived state that must be kept in memory. The plot separates three empirical claims. First, the RMSE labels stay near zero, confirming that the operator representation preserves the fitted L1 objective. Second, most datasets move left, showing faster fitting. Third, many points also move downward, showing lower peak resident memory. Points in the upper-left quadrant are still faster but use slightly more memory.

The two panels connect the separate timing and memory results. In the L1 panel in Figure 8, most datasets move left, and many also move downward, but the memory gains are smaller and less uniform than the timing gains. The L2 panel is shifted farther left and mostly downward because active-edge augmented-design construction reduces both the number of rows built before `glmnet` and the sparse objects held by the worker process. These patterns match the construction-cost analysis in Section 4. Across the L1 and L2 benchmarks, peak resident memory ranged from about 70 MB for featureless fits to about 500 MB for the largest full-pairwise reference fit.

7 Discussion

The experiments show that sparse fusion graphs need corresponding sparse computational representations. When the graph has m edges, an implementation that constructs $O(k^2)$ pairwise constraints can lose the computational advantage of using a sparse graph. This is a general issue for edge-penalized regression. Any method whose penalty decomposes over graph edges can pay an unnecessary quadratic cost if the graph is represented by a complete set of candidate pairs. The L1 operator and L2 active-edge builders recover the intended $O(pm)$ fusion representation while preserving the original objective in the exact cases.

The exact L1 operator is the safest default among the proposed L1 solvers. It preserves the same fitted objective as the reference active-edge formulation and gives substantial runtime and memory improvements. The chain-approximation solver is an approximation for exploratory or large-scale cases where a small loss in predictive accuracy is acceptable.

The solver representation aligns the computational graph with the modeling graph. Edge-explicit L1 fusion updates, blocked edge processing, DFS/MST chain approximation, and active-edge L2 matrix construction are instances of that representation. Proposition 1 gives the corresponding smoothed-objective convergence guarantee for the L1 APG updates, while Propositions 2 and 3 identify the exact cases where the new representations preserve the main objective. Proposition 4 clarifies how the graph construction cost changes with

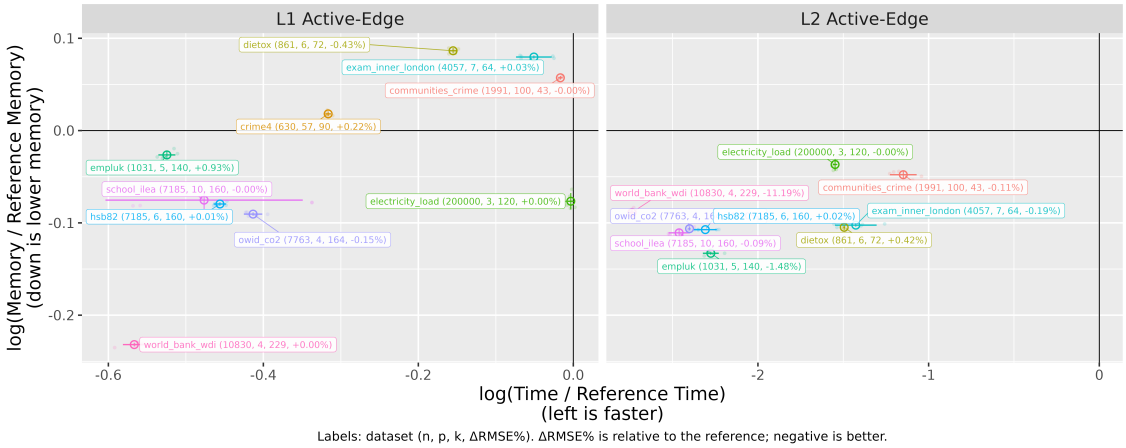

Figure 8: Runtime-memory Pareto plots for the exact active-edge solvers. The L1 panel compares the active-edge operator with the full-pairwise L1 reference, and the L2 panel compares active-edge augmented-design construction with the full-pairwise L2 reference. Left is faster, down is lower peak memory, and the lower-left quadrant indicates improvement on both efficiency axes.

the number of represented edges. The chain-approximation method is separated from those exact methods because replacing a graph by a chain changes the fusion geometry. Proposition 5 states the precise chain objective being optimized and the omitted-edge penalty that accounts for the approximation.

The empirical evidence is strongest for the exact L1 operator and active-edge L2 construction. The chain method provides a more aggressive efficiency option, but it changes the fusion graph before fitting. For sparse L1 fusion, the exact active-edge operator is therefore the preferred default, while chain-approximation fitting is best viewed as an option for exploratory or analyses with tight memory budgets.

7.1 When Sparse-Graph Representations Help

The sparse-graph representation is most useful when the modeling graph has far fewer active edges than the complete graph. The estimators considered in this paper allow arbitrary fusion graphs, including complete graphs. If the intended graph is complete, then $m = \binom{k}{2}$ and active-edge construction remains correct but does not reduce the asymptotic graph-representation cost. The computational gains therefore come from settings where the graph encodes a structured notion of similarity, such as temporal adjacency, spatial neighborhoods, nearest-neighbor group similarity, or task relationships. For complete or nearly complete graphs, the only L1 solver in this paper that can still reduce the fusion edge budget substantially is the chain approximation. It does so by replacing the original graph with a connected chain, so the gain comes from changing the optimization target rather than from an exact sparse representation of the same graph.

This distinction is separate from the statistical question of which graph is best for prediction. A complete graph may be appropriate when all group pairs should be eligible for fusion. In contrast, sparse graphs are appropriate when domain knowledge or training-fold similarity suggests that only a limited set of group relationships should be penalized directly. The contribution of the active-edge solvers is to make this sparse modeling choice visible to the optimizer.

7.2 Scope of the Optimization Comparisons

The experiments compare the new representations against the reference grouped fused-regression solvers rather than against a broad collection of alternative optimizer families. This choice is central to the empirical design. The L1 reference solver and the exact active-edge operator both use the same smoothed proximal-gradient framework. The L2 reference and active-edge solvers both use the same augmented-design reduction followed by `glmnet`. The comparisons therefore hold fixed the statistical objective, graph, folds, hyperparameter selection, prediction pipeline, and optimizer family. The primary changing factor is the representation of the graph-fusion term. This isolation makes the empirical results interpretable as evidence for the sparse-graph representation principle.

Alternative optimizers, including ADMM and split-Bregman methods, are important for fused-lasso problems and can be very effective in other settings. An ADMM formulation would typically introduce auxiliary variables for edge differences, enforce $z_{uv} = \beta_u - \beta_v$ with augmented-Lagrangian updates, alternate between coefficient, edge-variable, and dual updates, and require choices of penalty parameters and stopping rules. Such a comparison would mix representation effects with optimizer effects. Pre-conditioners, penalty-parameter tuning, convergence tolerances, warm starts and sparse-matrix backends could all influence runtime and memory. Those factors would not isolate the question studied here.

This distinction also clarifies the role of the reference fused-lasso baseline. It is included because it is the direct predecessor for the grouped fused-regression objective and software interface considered in this paper. Beating that baseline while preserving the same optimizer family shows that the gain comes from avoiding zero-weight all-pair graph construction rather than from replacing the optimization architecture. The same active-edge principle could be incorporated into ADMM or other edge-splitting methods, where the number of auxiliary edge variables and dual updates would also scale with the represented edge set. Developing and tuning such solvers is a separate algorithmic comparison and is left for future work.

There are two important limitations. First, the guarantees cover representation equivalence, smoothing error, convergence for the smoothed L1 objective, and graph-construction complexity. The statistical properties are those of the underlying fused-lasso and graph-guided regression objectives, while the contribution here is to preserve those objectives with a sparser computational representation. Second, the largest gains occur when the user-supplied or data-derived graph is sparse. If the modeling graph is close to complete, the active-edge representation has little to no advantage.

8 Implementation Details

The active-edge and chain approximation used for the sparse-graph comparisons are implemented entirely in R programming language. The main implementation challenge was to change the graph representation without changing the fitted objective. In the original all-pair construction, inactive graph pairs can still appear as zero-weight algebraic objects. This implies, looping over edges that do not exist, essentially wasting resources. Although, this is convenient for indexing, but it hides sparsity from the solver. The L1 operator code therefore first converts the fusion graph into edge arrays $(u_e, v_e, w_e)_{e=1}^{m}$ and keeps those arrays as the only fusion structure used inside the accelerated iterations. Each edge update computes a coefficient difference, clips the smoothed dual quantity, and scatters the result back to the two incident group columns with opposite signs. The difficult part was preserving the same orientation, weighting, intercept convention, group ordering, and final thresholding semantics as the reference implementation.

Memory management was another practical issue. A direct vectorized implementation can form a large edge by coefficient matrix during each L1 iteration. That is fast for moderate problems but can dominate memory when both p and m are large. The implementation supports blocked edge processing, which applies the same scatter updates in edge batches while keeping the accumulated gradient in the original coefficient matrix shape. The chain solver required a separate step. Specifically, groups are temporarily reordered along the selected chain for fast adjacent differences and then reordered back before returning fitted coefficients.

The L2 implementation required a different construction. The statistical problem is represented as an augmented least-squares design passed to `glmnet`. The key coding step was to build this augmented matrix directly from sparse triplets. For each active edge (g, h) and each fused coefficient coordinate, the builder emits one row with opposite signed entries in the two group blocks and a zero augmented response. This avoids allocating rows for inactive group pairs. The implementation also preserves the original loss scaling and lambda correction convention, so that any observed difference between full-pair and active-edge runs comes from the represented graph rows.

9 Software Availability

The algorithms are available in the open-source R package `sparsefusion` at `https://github.com/EngineerDanny/sparsefusion`. A grouped regression problem is supplied as a feature matrix X, response vector y, group labels `groups`, and a symmetric fusion graph matrix G. For new data, the recommended entry point is `sparse_fusion`. The argument `fusion` selects `l1` or `l2` fusion, and the argument `solver` selects `active_edge`, `full_pairwise`, or `chain_approx`. The `active_edge` solver is the exact sparse graph representation for L1 and L2 fusion, `full_pairwise` runs the reference all-pair construction, and `chain_approx` runs the approximate L1 chain solver. For example, an exact sparse-graph L1 fit on new grouped data can be run as follows:

```
library(sparsefusion)

fit <- sparse_fusion(
```

```
  X, y, groups, G,
  lambda = 1e-3,
  gamma = 1e-2,
  fusion = "l1",
  solver = "active_edge"
)
```

The returned object from each fitting function is a coefficient matrix with one column per group, so prediction on new observations requires selecting the coefficient column associated with the observation's group. The package includes examples, benchmark scripts, and a minimal reviewer example in the repository README. The paper experiments call the same package functions used for new data analysis, with fixed folds and graph construction scripts stored in the repository.

10 Conclusion

This paper studied the computational representation of sparse fusion graphs in grouped fused regression. The main finding is that a sparse statistical graph should remain sparse inside the optimizer. For L1 fusion, the active-edge operator evaluates the same smoothed fusion gradient as the full-edge construction after zero-weight pairs are removed. For L2 fusion, the active-edge augmented design gives the same quadratic fusion objective while constructing rows only for nonzero graph edges. These exact representations change the dominant fusion cost from all-pairs construction to construction that scales with the represented edge set. The empirical results support the same conclusion. The exact L1 and L2 active-edge solvers preserve predictive accuracy relative to the corresponding full-construction references while reducing fit time and, in many cases, peak memory. The chain approximation L1 solver gives an additional option when reducing the edge budget is more important than preserving the original graph exactly.

Several future directions remain open. First, the same active-edge representation principle could be incorporated into other optimizer families, including ADMM and split-Bregman methods, where auxiliary variables and dual updates should also scale with the represented edge set. Second, adaptive graph construction and hyperparameter selection could be studied jointly with the sparse solver representation, since the statistical value and computational cost of fusion both depend on the chosen graph. Third, lower-level sparse matrix and compiled implementations could further separate the algorithmic savings from fixed runtime overheads. These directions would test how far the representation principle extends beyond the grouped fused-regression solvers considered here. The algorithms used in this paper are implemented in the project repository at `https://github.com/EngineerDanny/sparsefusion`.

Acknowledgments and Disclosure of Funding

The author thanks the maintainers of the public datasets and R packages used in the empirical evaluation. The authors also acknowledge Northern Arizona University's high-performance computing resources for supporting the computational experiments. This work

was supported by National Science Foundation award 2125088 through the Rules of Life program. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author and do not necessarily reflect the views of the National Science Foundation.

References

Vincent Arel-Bundock. Rdatasets: A collection of datasets originally distributed in R packages, 2026. URL `https://vincentarelbundock.github.io/Rdatasets/`.

Amir Beck and Marc Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202, 2009.

Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011. doi: 10.1561/2200000016.

Xi Chen, Qihang Lin, Seyoung Kim, Jaime G. Carbonell, and Eric P. Xing. Graph-structured multi-task regression and an efficient optimization method for general fused lasso. In *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1–10, 2010.

Xi Chen, Qihang Lin, Seyoung Kim, Jaime G. Carbonell, and Eric P. Xing. Smoothing proximal gradient method for general structured sparse regression. *The Annals of Applied Statistics*, 6(2):719–752, 2012a.

Xiaohui Chen, Xinghua Shi, Xing Xu, Zhiyong Wang, Ryan Mills, Charles Lee, and Jinbo Xu. A two-graph guided multi-task lasso approach for eQTL mapping. In *Proceedings of the Fifteenth International Conference on Artificial Intelligence and Statistics*, volume 22 of *Proceedings of Machine Learning Research*, pages 208–217, 2012b.

Frank Dondelinger, Sach Mukherjee, and The Alzheimer's Disease Neuroimaging Initiative. The joint lasso: High-dimensional regression for group structured data. *Biostatistics*, 21 (2):219–235, 2020. doi: 10.1093/biostatistics/kxy035.

Jerome Friedman, Trevor Hastie, and Robert Tibshirani. Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1):1–22, 2010.

Andrew Gelman and Jennifer Hill. *Data Analysis Using Regression and Multilevel/Hierarchical Models*. Cambridge University Press, Cambridge, 2007.

David Hallac, Jure Leskovec, and Stephen Boyd. Network lasso: Clustering and optimization in large graphs. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 387–396, 2015.

Holger Hoefling. A path algorithm for the fused lasso signal approximator. *Journal of Computational and Graphical Statistics*, 19(4):984–1006, 2010.

Ron Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence*, pages 1137–1143. Morgan Kaufmann, 1995.

Oscar Hernan Madrid Padilla, James Sharpnack, James G. Scott, and Ryan J. Tibshirani. The dfs fused lasso: Linear-time denoising over general graphs. *Journal of Machine Learning Research*, 18(176):1–36, 2018. URL `https://www.jmlr.org/papers/v18/16-532.html`.

Oscar Hernan Madrid Padilla, James Sharpnack, Yanzhen Chen, and Daniela M. Witten. Adaptive nonparametric regression with the K-nearest neighbour fused lasso. *Biometrika*, 107(2):293–310, 2020. doi: 10.1093/biomet/asz071.

Yurii Nesterov. Smooth minimization of non-smooth functions. *Mathematical Programming*, 103(1):127–152, 2005.

Northern Arizona University Advanced Research Computing. Overview of monsoon high-performance computing resources, 2026. URL `https://in.nau.edu/arc/overview/`. Accessed 2026-05-28.

Nooshin Omranian, Jeanne M. O. Eloundou-Mbebi, Bernd Mueller-Roeber, and Zoran Nikoloski. Gene regulatory network inference using fused LASSO on multiple data sets. *Scientific Reports*, 6:20533, 2016. doi: 10.1038/srep20533.

Oscar Hernan Madrid Padilla, James Sharpnack, James G. Scott, and Ryan J. Tibshirani. Adaptive nonparametric regression with the K-nearest neighbour fused lasso. *Biometrika*, 107(2):293–310, 2020.

Ashley Petersen, Daniela Witten, and Noah Simon. Fused lasso additive model. *Journal of Computational and Graphical Statistics*, 25(4):1005–1025, 2016.

José C. Pinheiro and Douglas M. Bates. *Mixed-Effects Models in S and S-PLUS*. Springer, New York, 2000. doi: 10.1007/b98882.

Stephen W. Raudenbush and Anthony S. Bryk. *Hierarchical Linear Models: Applications and Data Analysis Methods*. Sage Publications, Thousand Oaks, CA, 2 edition, 2002.

Michael Redmond. Communities and crime. UCI Machine Learning Repository, 2002. DOI: 10.24432/C53W3X.

Hannah Ritchie, Pablo Rosado, and Max Roser. CO2 and greenhouse gas emissions. Our World in Data, 2023.

Wesley Tansey, Yu-Xiang Wang, David M. Blei, and Raul Rabadan. The DFS fused lasso: Linear-time denoising over general graphs. *Journal of Machine Learning Research*, 19 (176):1–36, 2018.

Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1):267–288, 1996.

Robert Tibshirani, Michael Saunders, Saharon Rosset, Ji Zhu, and Keith Knight. Sparsity and smoothness via the fused lasso. *Journal of the Royal Statistical Society: Series B*, 67 (1):91–108, 2005.

Ryan J. Tibshirani and Jonathan Taylor. The solution path of the generalized lasso. *The Annals of Statistics*, 39(3):1335–1371, 2011.

Artur Trindade. Electricityloaddiagrams20112014. UCI Machine Learning Repository, 2015. DOI: 10.24432/C58C86.

Sudhir Varma and Richard Simon. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7:91, 2006. doi: 10.1186/1471-2105-7-91.

Yu-Xiang Wang, James Sharpnack, Alexander J. Smola, and Ryan J. Tibshirani. Trend filtering on graphs. *Journal of Machine Learning Research*, 17(105):1–41, 2016.

World Bank. World development indicators. World Bank DataBank, 2026.

Gui-Bo Ye and Xiaohui Xie. Split bregman method for large scale fused lasso. *Computational Statistics & Data Analysis*, 55(4):1552–1569, 2011. doi: 10.1016/j.csda.2010.10.021.

Feng Yu, Archer Yi Yang, and Teng Zhang. A graph decomposition-based approach for the graph-fused lasso. *Journal of Statistical Planning and Inference*, 235:106221, 2025. doi: 10.1016/j.jspi.2024.106221.

Ming Yuan and Yi Lin. Model selection and estimation in regression with grouped variables. *Journal of the Royal Statistical Society: Series B*, 68(1):49–67, 2006.

Hui Zou and Trevor Hastie. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B*, 67(2):301–320, 2005. doi: 10.1111/j.1467-9868.2005.00503.x.